VIET NAM NATIONAL UNIVERSITY HO CHI MINH

UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

Shortest Path Algorithms

Seminar

Course: Data Structures - CS163

Student list:

Nguyễn Anh Kiệt (25125074)

Trần Như Khải (25125045)

Nguyễn Đức Huy (25125014)

Nguyễn Đình Minh Huy (25125083)

Group 05 - Class 25A01

18th April 2026

Contents

1 Introduction & History **2**

2 Main Data Structures & Algorithms **2**

3 Single Source Shortest Path (SSSP) **2**

3.1 Shortest Path in DAGs . . . 2

3.1.1 Topological Order . . . 2

3.1.2 DAG Shortest Path Processing . . . 3

3.2 Dijkstra's Algorithm . . . 4

3.2.1 Dijkstra Optimizing . . . 5

3.3 Shortest Path in Bellman-Ford . . . 6

3.3.1 How Bellman-Ford works: Global Relaxation . . . 6

3.3.2 Bellman-Ford Superpower: Negative Cycle Detection . . . 7

3.4 SSSP Algorithm: How to track the path? . . . 9

4 All Pairs Shortest Path (APSP) **9**

4.1 Floyd-Warshall Algorithm . . . 9

4.1.1 The Intermediate Node . . . 10

4.1.2 How to track the path? . . . 11

5 Step-by-step Demo **11**

6 Implementation Notes & Complexity **11**

6.1 Implementation Notes . . . 11

6.2 Complexity Notes in Practice . . . 12

6.3 Engineering Trade-offs . . . 12

7 Applications **13**

8 Quiz **13**

9 Programming Assignments **13**

10 Source Codes & Instructions **13**

10.1 Dependencies 13

10.2 What Is Included 13

10.3 Input Graph Format 14

10.4 Compile Instructions (Maximum Optimization) 14

10.4.1 macOS (clang++) 14

10.4.2 Windows (MSYS2/MinGW-w64 g++) 14

10.4.3 Windows (MSVC cl) 15

10.5 Run Workflow (After Compilation) 15

10.6 Generate the Benchmark Datasets 17

10.7 Generate the DAG Benchmark Dataset 17

11 Datasets **17**

11.1 Dataset Suite 17

11.2 Dataset Files 18

11.3 Header Format 18

11.4 Benchmark Interpretation 18

12 Comparison **18**

References **19**

Group Information

This report is conducted by Group 05. Below is the list of group members:

Student ID	Full Name
25125074	Nguyễn Anh Kiệt
25125045	Trần Như Khải
25125014	Nguyễn Đức Huy
25125083	Nguyễn Đình Minh Huy

1 Introduction & History

2 Main Data Structures & Algorithms

In a weighted graph, the objective of a shortest path algorithm is to identify a sequence of edges connecting vertices such that the sum of the edge weights is minimized. The problem is generally bifurcated into two primary categories:

- **Single Source Shortest Path (SSSP):** Determines the minimum cost to travel from a single starting node (the source) to all other reachable nodes in the graph.
- **All Pairs Shortest Path (APSP):** Computes the shortest paths between every possible pair of vertices simultaneously.

3 Single Source Shortest Path (SSSP)

The Single Source Shortest Path (SSSP) problem asks a fundamental question: *What is the shortest path from one starting node to all other nodes?*

3.1 Shortest Path in DAGs

Historical Context: While shortest path traversal on Directed Acyclic Graphs represents a fundamental application of graph theory rather than a single named algorithm, its core mechanism—Topological Sorting—was mathematically formalized by Arthur B. Kahn in 1962. The sequential evaluation method used to find shortest paths has direct roots in the PERT (Program Evaluation and Review Technique) charts developed by the United States Navy in the 1950s for project scheduling.

A **Directed Acyclic Graph (DAG)** is a directed graph which forms no cycles or loops. The SSSP problem can be solved efficiently on a DAG due to the fact that the nodes can be ordered in a topological ordering via topsort and processed sequentially.

3.1.1 Topological Order

A *linear ordering of its vertices such that for every directed edge from vertex u to vertex v ($u \to v$), vertex u comes before vertex v in the ordering.*

Topological sorting can be implemented in two standard ways: a DFS-based approach or a BFS-based approach (Kahn's algorithm). Kahn's algorithm is especially useful when we want built-in DAG validation, because it can detect non-DAG inputs when not all vertices can be processed. In this implementation, we use the DFS-based method and assume the input graph is already a DAG.

Topological Sort Pseudocode:

Algorithm 1 Topological Sort

```
function TOPOLOGICALSORT(graph)
    visited ← [false, false, ..., false]                ▷ size V
    order ← empty stack
    for i ← 0 to V − 1 do
        if not visited[i] then
            DFS(graph, i, visited, order)
        end if
    end for
    return order
end function
function DFS(graph, index, visited, order)
    visited[index] ← true
    for edge in graph[index] do
        if not visited[edge.to] then
            DFS(graph, edge.to, visited, order)
        end if
    end for
    order.push(index)
end function
```

3.1.2 DAG Shortest Path Processing

1. Select the first coming vertex from the topological ordering.
2. Explore all its neighbours.
3. If finding a shorter path, relax the edge.
4. Reprocess until finishing.

DAG Shortest Path Pseudocode:

Algorithm 2 DAG Shortest Path

```
function dagShortestPath(graph, start)
    distances ← [∞, ∞, ..., ∞]                    ▷ size V
    distances[start] ← 0
    topoOrder ← topologicalSort(graph)
    while topoOrder is not empty do
        index ← topoOrder.pop()
        if distances[index] == ∞ then
            continue
        end if
        for edge in graph[index] do
            newDist ← distances[index] + edge.cost
            if newDist < distances[edge.to] then
                distances[edge.to] ← newDist
            end if
        end for
    end while
    return distances
end function
```

Complexity Analysis:

- **Time Complexity: O(V + E)**. We are traversing every vertex and edge.
- **Space Complexity: O(V)**. We are storing the topological ordering and the distances array.

3.2 Dijkstra's Algorithm

Historical Context: Conceived by the Dutch computer scientist Edsger W. Dijkstra in 1956 (and formally published in 1959 in *A note on two problems in connexion with graphs*), this algorithm was famously designed in just twenty minutes while Dijkstra was drinking coffee at a café in Amsterdam. He initially created it to find the shortest path between the cities of Rotterdam and Groningen to demonstrate the capabilities of the new ARMAC computer.

Given a graph of non-negative weight edges. We can greedily assume that adding edges can only increase the total path cost.

Therefore, we come up with the **Greedy strategy**: continually select the unvisited node with the smallest known distance using a min-heap assuming it cannot be improved anymore, and then relaxes its outgoing edges.

Shortest Path Pseudocode:

Algorithm 3 Dijkstra's Algorithm

```
function DIJKSTRA(graph, start)
    visited ← [false, false, ..., false]                ▷ size V
    distances ← [∞, ∞, ..., ∞]                          ▷ size V
    distances[start] ← 0
    pq ← empty priority queue (min-heap)
    pq.insert({0, start})
    while pq is not empty do
        index, minValue ← pq.poll()
        visited[index] ← true
        if distances[index] < minValue then
            continue
        end if
        for edge in graph[index] do
            if visited[edge.to] then
                continue
            end if
            newDist ← distances[index] + edge.cost
            if newDist < distances[edge.to] then
                distances[edge.to] ← newDist
                pq.insert({edge.to, newDist})
            end if
        end for
    end while
    return distances
end function
```

Complexity Analysis:

- **Time Complexity:** $\mathbf{O}((\mathbf{V}+\mathbf{E})\log\mathbf{V})$. We extract exactly V vertices. Each extraction from a heap takes $O(\log V)$. We evaluate at most E edges. If we relax the edge, we push new value into the heap takes $O(\log V)$.
- **Space Complexity:** $\mathbf{O}(\mathbf{V}+\mathbf{E})$. We are storing the distances array for every vertex and the priority queue for each relaxed edge.

3.2.1 Dijkstra Optimizing

- **Lazy Dijkstra:** uses a standard Binary Heap. Time complexity: $\mathbf{O}((\mathbf{V}+\mathbf{E})\log\mathbf{V})$.
- **Eager Dijkstra:** uses an indexed priority queue to instantly locate the vertex and update the value dynamically. Time complexity: $\mathbf{O}((\mathbf{V}+\mathbf{E})\log\mathbf{V})$.

- **D-ary Heap Dijkstra (dense graph optimizer):** uses a Heap giving each parent D children, cheaper when updating but more expensive when polling.
  Time complexity: $\mathbf{O(V \log_D V + E \log_D V)}$.

- **Fibonacci Heap Dijkstra:** Replaces the standard tree-based heap with a highly complex collection of trees called a Fibonacci Heap which achieved updating time an amortized $O(1)$.
  Time Complexity: $\mathbf{O(V \log V + E)}$.

3.3 Shortest Path in Bellman-Ford

Historical Context: This algorithm primarily bears the names of Richard Bellman (who published it in 1958 in *On a routing problem*) and Lester R. Ford Jr. (who developed the foundational flow formulations in 1956). However, the algorithm was independently published by Edward F. Moore in 1959 and Alfonso Shimbel in 1955. As a pioneering application of Bellman's principle of optimality, it introduced dynamic programming to the realm of graph routing.

When a graph contains negative weight edges, Dijkstra's greedy logic break down. What should we do?
Bellman-Ford is a Single Source Shortest Path (SSSP) algorithm designed specifically to handle graphs with negative weights. Unlike Dijkstra (Greedy), Bellman-Ford uses **Dynamic Programming**. It calculates shortest paths in a bottom-up manner by gradually allowing paths to use more and more edges.

3.3.1 How Bellman-Ford works: Global Relaxation

- **Step 1: Initialization.** Set the distance to the source node to 0 and all other nodes to ∞.

- **Step 2: Edge Relaxation.** Iterate through every single edge (u, v) with weight w in the graph. If a new shorter path is found, then relax that edge.

- **Step 3: Repeat.** Perform this full-graph relaxation exactly $\mathbf{V - 1}$ **times**.

Why V - 1 times? We expect after one loop the path is extended by at least one edge. The shortest path in a graph with V vertices without cycles, contains at least $V - 1$ edges.

3.3.2 Bellman-Ford Superpower: Negative Cycle Detection

Negative cycle: *A loop in the graph where the sum of the edge weights is less than zero. If one exists, you can loop infinitely to achieve a distance of $-\infty$. No shortest path can be defined.*

After running $V - 1$ iterations, run the relaxation one more time. If any edge still can be relaxed, that edge is in a negative cycle. All vertices can be routed from the negative vertices have $-\infty$ distance.

Bellman-Ford Pseudocode:

Algorithm 4 Bellman-Ford Algorithm

```
function BELLMANFORD(graph, start)
    dist ← [∞, ∞, ..., ∞]                                   ▷ size V
    dist[start] ← 0
    for i ← 1 to V − 1 do
        for edge in graph.edges do
            if dist[edge.from] + edge.cost < dist[edge.to] then
                dist[edge.to] ← dist[edge.from] + edge.cost
            end if
        end for
    end for
    negVertices ← empty queue
    for edge in graph.edges do
        if dist[edge.from] + edge.cost < dist[edge.to] then
            dist[edge.to] ← −∞
            negVertices.enqueue(edge.to)
        end if
    end for
    PROCESSNEGATIVECYCLE(graph, negVertices, dist)
    return dist
end function
function PROCESSNEGATIVECYCLE(graph, negVertices, dist)
    visited ← [false, false, ..., false]                    ▷ size V
    while negVertices is not empty do
        vertex ← negVertices.dequeue()
        if visited[vertex] then
            continue
        end if
        DFS(graph, vertex, negVertices, dist, visited)
    end while
end function
function DFS(graph, vertex, negVertices, dist, visited)
    visited[vertex] ← true
    for edge in graph[vertex] do
        if visited[edge.to] then
            continue
        end if
        dist[edge.to] ← −∞
        negVertices.enqueue(edge.to)
        DFS(graph, edge.to, negVertices, dist, visited)
    end for
end function
```

Complexity Analysis:

- **Time Complexity: $\mathbf{O(E \cdot V)}$**. We iterate every edges $V - 1$ times and one more loop for negative cycle detection.
- **Space Complexity: $\mathbf{O(V)}$**. We are storing the distances array for every vertex and the priority queue for each relaxed edge.

Data Insight (Placeholder):

on the dataset.

3.4 SSSP Algorithm: How to track the path?

By introducing a `prev` array, where `prev[i]` denotes its previous vertex. We simply update it whenever relaxing the edges:

```
if dist[edge.from] + edge.cost < dist[edge.to] then
    dist[edge.to] ← dist[edge.from] + edge.cost
    prev[edge.to] ← edge.from
end if
```

Then when we need to track the path, we can route from the end to start through the `prev` array then reversing that result.

4 All Pairs Shortest Path (APSP)

What is the shortest path between all pairs of nodes?

4.1 Floyd-Warshall Algorithm

Historical Context: Published independently by Robert W. Floyd (*Algorithm 97: Shortest Path*, 1962) and Stephen Warshall (*A theorem on Boolean matrices*, 1962), this algorithm is an elegant realization of dynamic programming. It fundamentally relies on a theorem Warshall initially developed for computing the transitive closure of a graph, which Floyd ingeniously adapted to compute minimal path weights.

We are no longer looking for paths from a single source. We want to find the shortest path between every pair of vertices (u, v) simultaneously.

Floyd-Warshall is an APSP dynamic programming algorithm which relies on the idea of comparing between direct connection and routing through an intermediate vertex.

4.1.1 The Intermediate Node

Can we find a shorter path from i to j by routing it through an intermediate node k?

- **Initialization:** Create a 2D matrix where `dist[i][j]` is the weight of the direct edge from i to j (or ∞ if no direct edge exists).
- We iterate V loops with index of k, that $\texttt{dist}[i][k] + \texttt{dist}[k][j]$ is the distance when routing through vertex k. If it is better than the current connection, we update $\texttt{dist}[i][j]$. We are processing for every pair (i, j).

Floyd-Warshall Pseudocode:

Algorithm 5 Floyd-Warshall Algorithm

```
function SETUP(graph, dist)
    for i ← 0 to V − 1 do
        dist[i][i] ← 0
    end for
    for edge in graph.edges do
        dist[edge.from][edge.to] ← edge.cost
    end for
end function
function FLOYDWARSHALL(graph)
    dist ← 2D array of size V × V initialized to ∞
    SETUP(graph, dist)
    for k ← 0 to V − 1 do
        for i ← 0 to V − 1 do
            for j ← 0 to V − 1 do
                if dist[i][k] + dist[k][j] < dist[i][j] then
                    dist[i][j] ← dist[i][k] + dist[k][j]
                end if
            end for
        end for
    end for
    return dist
end function
```

Complexity Analysis:

- **Time Complexity: $\mathbf{O(V^3)}$**. We iterate V loops to update for every pair of (i, j).
- **Space Complexity: $\mathbf{O(V^2)}$**. We only use a 2D matrix to store the distance between every pair of vertices.

4.1.2 How to track the path?

By introducing a `next` 2D matrix, where `next[i][j]` denotes the next node from i when routing to j. We initialize `next[i][j] = j` if there is a direct connection or `next[i][j] = -1`. After initializing, in the main loop, we update the `next` matrix as well as the distance:

```
if dist[i][k] + dist[k][j] < dist[i][j] then
    dist[i][j] ← dist[i][k] + dist[k][j]
    next[i][j] ← k
end if
```

Then when we need to track the path, we can route from the start to end through the `next` matrix.

5 Step-by-step Demo

6 Implementation Notes & Complexity

6.1 Implementation Notes

Our implementation is organized around a common graph interface (`IGraphStructure`) and multiple concrete graph representations.

- **Graph abstraction:** `IGraphStructure` provides a unified API so all shortest path methods can run on either adjacency list or adjacency matrix.
- **Heap implementation update:** Dijkstra-related methods now rely on dedicated heap headers in `SourceCode/Heap/`, including `IndexedPriorityQueue` (eager Dijkstra) and `IndexedDaryHeap` (D-ary optimized Dijkstra).
- **Primary runtime choice:** Benchmarking is executed on `AdjacencyList` because the provided datasets are sparse to medium-density and edge traversal dominates runtime.

- **Two-phase loading for large files:** The loader first scans the file to estimate vertex range and metadata, then performs a second pass to build the graph. This avoids repeated graph resizing during insertion.
- **Input robustness:** Each edge line can be either `u v` or `u v w`. Missing weights default to 1.
- **Practical runtime controls:** Floyd-Warshall is run on a bounded induced subgraph (first k vertices), and heavy algorithms can be skipped via command flags for large-scale datasets.

6.2 Complexity Notes in Practice

Theoretical complexities are summarized in Section 2. In implementation, the observed behavior is consistent with those bounds:

- **Dijkstra variants:** Fast on non-negative sparse graphs due to heap-based relaxations (binary heap, indexed priority queue, and indexed D-ary heap).
- **Bellman-Ford:** Significantly heavier for large edge sets because each full relaxation pass touches all edges.
- **Floyd-Warshall:** Cubic in vertices, therefore limited to a bounded subset for benchmark practicality.

6.3 Engineering Trade-offs

- **Adjacency List vs Matrix:** Adjacency list reduces memory for sparse graphs and is preferred for SSSP workloads; adjacency matrix is simpler for APSP semantics but grows as $O(V^2)$ in memory.
- **Benchmark fidelity vs runtime budget:** We keep full algorithm coverage, but expose skip options so seminar demonstrations remain responsive.

7 Applications

8 Quiz

9 Programming Assignments

10 Source Codes & Instructions

The source files are in `SourceCode/ShortestPath/`, while heap headers are in `SourceCode/Heap/`.

10.1 Dependencies

The benchmark now supports an optional Boost-backed heap backend. On macOS, install it with:

```
brew install boost
```

If Boost headers are unavailable, the benchmark automatically falls back to the custom heap implementations.

10.2 What Is Included

- Core graph structures: `AdjacencyList.*`, `AdjacencyMatrix.*`, `IGraphStructure.h`, `Edge.h`.
- Shortest path implementations: `ShortestPath.*`.
- Benchmark runner source: `read_graph_info.cpp`.
- Graph generator source: `graph_generator.cpp`.
- DAG graph generator source: `dag_graph_generator.cpp`.
- Heap implementations: `IndexedPriorityQueue.h`, `IndexedDaryHeap.h`, `BinaryHeap.h`.
- Datasets: `dataset1_sparse_large.txt`, `dataset2_dense_medium.txt`, `dataset3_complete_apsp.txt` `dataset4_negative_sssp.txt`, `dataset5_dag_sssp.txt`.
- Benchmark logs: `bench_dataset1_sparse_large_boost.log` to `bench_dataset5_dag_sssp_boost.log`

10.3 Input Graph Format

Each edge line is:

```
from to weight
```

If `weight` is omitted, the loader defaults to weight 1.

10.4 Compile Instructions (Maximum Optimization)

All commands below are compile-only and use aggressive optimization settings.

Working directory (from repository root):

```
cd SourceCode/ShortestPath
```

10.4.1 macOS (clang++)

Compile graph generator:

```
clang++ -std=c++20 -O3 -march=native -mtune=native -flto \
  -DNDEBUG graph_generator.cpp -o graph_generator
```

Compile DAG graph generator:

```
clang++ -std=c++20 -O3 -march=native -mtune=native -flto \
  -DNDEBUG dag_graph_generator.cpp -o dag_graph_generator
```

Compile benchmark runner:

```
clang++ -std=c++20 -O3 -march=native -mtune=native -flto \
  -DNDEBUG -I/opt/homebrew/include read_graph_info.cpp ShortestPath.cpp \
  AdjacencyList.cpp AdjacencyMatrix.cpp -o benchmark
```

10.4.2 Windows (MSYS2/MinGW-w64 g++)

Compile graph generator:

```
g++ -std=c++20 -O3 -march=native -mtune=native -flto \
  -DNDEBUG graph_generator.cpp -o graph_generator.exe
```

Compile DAG graph generator:

```
g++ -std=c++20 -O3 -march=native -mtune=native -flto \
  -DNDEBUG dag_graph_generator.cpp -o dag_graph_generator.exe
```

Compile benchmark runner:

```
g++ -std=c++20 -O3 -march=native -mtune=native -flto \
  -DNDEBUG read_graph_info.cpp ShortestPath.cpp \
  AdjacencyList.cpp AdjacencyMatrix.cpp -o benchmark.exe
```

10.4.3 Windows (MSVC cl)

Compile graph generator:

```
cl /std:c++20 /O2 /GL /DNDEBUG graph_generator.cpp \
  /link /LTCG /OUT:graph_generator.exe
```

Compile DAG graph generator:

```
cl /std:c++20 /O2 /GL /DNDEBUG dag_graph_generator.cpp \
  /link /LTCG /OUT:dag_graph_generator.exe
```

Compile benchmark runner:

```
cl /std:c++20 /O2 /GL /DNDEBUG read_graph_info.cpp ShortestPath.cpp \
  AdjacencyList.cpp AdjacencyMatrix.cpp /link /LTCG /OUT:benchmark.exe
```

Note: MSVC does not support `-march=native`. The closest equivalent is `/O2` with link-time optimization (`/GL + /LTCG`).

10.5 Run Workflow (After Compilation)

1. Generate a sample graph:

```
# macOS / Linux
./graph_generator 10000 50000 1 20 1 demo_graph.txt

# Windows (MinGW or MSVC shell)
graph_generator.exe 10000 50000 1 20 1 demo_graph.txt
```

Arguments:

- `10000`: number of vertices.
- `50000`: number of edges.
- `1 20`: minimum and maximum edge weights.
- `1`: directed graph (use `0` for undirected).
- `demo_graph.txt`: output filename.

2. Run benchmark:

```
# macOS / Linux
./benchmark dataset1_sparse_large.txt 3 200 --heap-backend=boost


# Windows (MinGW or MSVC shell)
benchmark.exe dataset1_sparse_large.txt 3 200 --heap-backend=boost
```

Benchmark positional arguments:

- `dataset1_sparse_large.txt`: input graph file path.
- `3`: number of source vertices used for SSSP runs.
- `200`: Floyd-Warshall vertex cap (induced subgraph size).

Benchmark flags (detailed):

- `-skip-bellman`: skips Bellman-Ford only. Use this for large graphs when negative-edge support is not being tested.
- `-skip-floyd`: skips Floyd-Warshall only. Use this when APSP is too expensive for the selected graph size.
- `-skip-heavy`: skips both Bellman-Ford and Floyd-Warshall. Recommended for quick demo runs focused on Dijkstra variants.
- `-heap-backend=custom`: uses the custom indexed heap implementations.

- `-heap-backend=boost`: uses Boost heap containers when Boost headers are installed.

Example full run (no skipping, small graph):

```
./benchmark dataset4_negative_sssp.txt 3 200
```

DAG benchmark note: for DAG inputs, the benchmark reports both the original `dagShortestPath` implementation and the fairer `dagShortestPath (precomputed topo order)` variant, which reuses one topological order across source runs.

10.6 Generate the Benchmark Datasets

```
./graph_generator 100000 500000 1 10000 1 dataset1_sparse_large.txt
./graph_generator 5000 10000000 1 100000 1 dataset2_dense_medium.txt
./graph_generator 1000 999000 1 1000 1 dataset3_complete_apsp.txt
./graph_generator 1000 25000 -100 1000 1 dataset4_negative_sssp.txt
```

10.7 Generate the DAG Benchmark Dataset

```
./dag_graph_generator 100000 500000 1 10000 1 dataset5_dag_sssp.txt
```

11 Datasets

We generated five directed weighted benchmark datasets to stress different shortest path regimes.

11.1 Dataset Suite

Test Dataset	**Vertices** (V)	**Edges** (E)	**Min W** (m)	**Max W** (M)	**Primary Algorithms t**
Sparse Large	100,000	500,000	1	10,000	Lazy vs. Eager vs. D-ary
Dense Medium	5,000	10,000,000	1	100,000	Eager vs. D-ary vs. Belln
Complete (APSP)	1,000	999,000	1	1,000	Floyd-Warshall vs. ($V\times$
Negative SSSP	1,000	25,000	-100	1,000	Bellman-Ford vs. Floyd-V
Dag SSSP	100,000	500,000	1	10,000	DAG shortest path vs. D

11.2 Dataset Files

- `dataset1_sparse_large.txt`
- `dataset2_dense_medium.txt`
- `dataset3_complete_apsp.txt`
- `dataset4_negative_sssp.txt`
- `dataset5_dag_sssp.txt`

11.3 Header Format

Each dataset is emitted with metadata header lines, where the dataset name is taken from the output filename:

```
# Directed <dataset_name> graph dataset
# Nodes: <V> Edges: <E>
# FromNodeId    ToNodeId
```

followed by weighted edges in the form:

```
from to weight
```

11.4 Benchmark Interpretation

The DAG dataset is important because it highlights two different costs:

- `dagShortestPath`: recomputes the topological traversal each run.
- `dagShortestPath (precomputed topo order)`: reuses one topological order across source runs and is the fairer textbook-style comparison.

12 Comparison

References

[1] Dan Bernstein. Hash functions (djb2) - comp.lang.c usenet, 1991. Accessed: 2025-12-28.

[2] Eraser.io. Eraser diagram tool. https://eraser.io/, 2025. Online tool used to design UML diagrams and system architecture illustrations.

[3] Google. Gemini. https://gemini.google.com/, 2025. Generative AI model used to support progress tracking, multithreading concepts, cross-platform implementation, code refactoring, and report writing.

[4] OpenAI. Chatgpt. https://chat.openai.com/, 2025. AI assistant used to help draft technical reports, convert content to LaTeX, and format tables, diagrams, and documentation based on student-provided input.